Octave code for the wrapper (1)

- In this lesson, you will learn how to implement the SPKF parameter estimator in Octave, and will see results of applying it to the linear spring-mass-damper model.
  - Note that even though the model is linear, we still require a nonlinear KF to perform parameter estimation.
  - The unknown parameters are: $\theta = [k, b, m]^T$.
- The Octave wrapper code loads data and reserves storage for results.

```
% Load data file comprising model parameters and simulation profile
load simOut.mat model dT t u x z

% Reserve storage for computed results, for plotting
[nx,nt]    = size(x);
nq         = 3; % this many parameters to estimate
qhatstore  = zeros(nq,nt);
boundstore = zeros(nq,nt);
```


Octave code for the wrapper (2)

- The wrapper code then defines true parameters q, defines our initial guess of θ, its covariance, and some noise matrices. Then, it initializes the SPFK data structures.

```
% Covariance values
q       = [model.k, model.b, model.m]';
q0      = [2;1;40];
SigmaQ0 = diag((q-q0)/3)^2; % uncertainty of initial parameters
SigmaW  = diag(1e-8*[1 1 10]);
SigmaD  = blkdiag(model.SigmaW,model.SigmaV);

% Create spkfData structure and initialize variables
spkfData = initSPKFid(q0,SigmaQ0,SigmaW,SigmaD,u(1),x(:,1));
```


Octave code for the wrapper (3)

- The main loop is very similar to the one used in a SPKF for state estimation.

```
% Now, enter loop for remainder of time, where we update the SPKF
% once per sample interval
for k = 2:length(t)
  uk = u(k);   % "measure" input force
  xk = x(:,k); % "measure" true state
  zk = z(k);   % "measure" nonlinear output

  % Update parameters
  [qhatstore(:,k),boundstore(:,k),spkfData] = iterSPKFid(uk,xk,zk,dT,spkfData);
end
```

- Plotting code is also very similar, so I have omitted it from this lesson.

Octave code for `initSPKFid`

- The `initSPKFid` function stores parameters in `spkfData`.

```
function spkfData = initSPKFid(q0,SigmaQ0,SigmaW,SigmaD,u1,x1)
  spkfData.qhat   = q0;       % Initial state description
  spkfData.SigmaQ = SigmaQ0;  % Initial estimation-error covariance
  spkfData.SigmaD = SigmaD;   % Sensor-noise covariance
  spkfData.SigmaW = SigmaW;   % Process-noise covariance
  spkfData.priorU = u1;       % Previous value of input force
  spkfData.priorX = x1;       % Previous value of state
  spkfData.Qbump  = 5;        % In case SigmaQ collapses
  spkfData.Snoise = real(chol(spkfData.SigmaD,'lower'));

  % SPKF specific parameters
  nq = length(spkfData.qhat); spkfData.nq = nq; % param-vector length
  nd = size(SigmaD,1); spkfData.nd = nd;        % meas-vector length
  nu = 1; spkfData.nu = nu;                     % input-vector length
  nw = size(SigmaQ0,1); spkfData.nw = nw; % process-noise-vector length
  nv = size(SigmaD,1); spkfData.nv = nv;  % sensor-noise-vector length
  na = nq+nd; spkfData.na = na;           % augmented-param-vector length
```

- Initialization of h, $\alpha^{(\mathrm{m})}$, and $\alpha^{(\mathrm{c})}$ is same as before, so I have omitted it here.

Octave code for `iterSPKFid` (1)

- The iteration code in `iterSPKFid` first unpacks parameters from `spkfData` :

```
function [qhat,bounds,spkfData] = iterSPKFid(uk,xk,zk,dT,spkfData)
  SQRT = @(x) sqrt(max(0,x)); % "Safe" square root

  % Get data stored in spkfData structure
  priorU = spkfData.priorU;
  priorX = spkfData.priorX;
  SigmaQ = spkfData.SigmaQ;
  SigmaW = spkfData.SigmaW;
  qhat = spkfData.qhat;
  nq = spkfData.nq;
  nv = spkfData.nv;
  na = spkfData.na;
  Snoise = spkfData.Snoise;
  Wc = spkfData.Wc;
```


Octave code for `iterSPKFid` (2)

- Then, it executes step 1a to the first part of 1c.

```
  % Step 1a: Parameter prediction time update
  %          - Do nothing. qhat(minus,k) = qhat(plus,k-1)

  % Step 1b: Prediction-error covariance time update
  SigmaQ = SigmaQ + SigmaW;

  % Step 1c: Output prediction
  % Step 1c-1: Create augmented SigmaQ and qhat
  [sigmaQa,p] = chol(SigmaQ,'lower');
  if p>0
    fprintf('Cholesky error.  Recovering...\n');
    theAbsDiag = abs(diag(SigmaQ));
    sigmaQa = blkdiag(max(SQRT(theAbsDiag),SQRT(spkfData.SigmaW)));
  end
  sigmaQa=blkdiag(real(sigmaQa),Snoise);
  qhata = [qhat; zeros([nv 1])];
  % NOTE: sigmaQa is lower-triangular
```

Octave code for `iterSPKFid` (3)

- It continues with 1c–2a.

```
% Step 1c-2: Calculate SigmaQ points (strange indexing of
% qhata to avoid "repmat" call, which is very inefficient)
Qa = qhata(:,ones([1 2*na+1])) + ...
     spkfData.h*[zeros([na 1]), sigmaQa, -sigmaQa];

% Step 1c-3: Compute prediction of [xk;zk]
D = outputEqn(Qa(1:nq,:),priorU,xk,priorX,dT,Qa(nq+1:end,:));
dhat = D*spkfData.Wm; % prediction of [xk;zk]

% Step 2a: Estimator gain matrix
Qs = Qa(1:nq,:) - qhat(:,ones([1 2*na+1]));
Ds = D - dhat(:,ones([1 2*na+1]));
SigmaQD = Qs*diag(Wc)*Ds';
SigmaD  = Ds*diag(Wc)*Ds';
L = SigmaQD/SigmaD;
```


Octave code for `iterSPKFid` (4)

- It continues with 2b, saves results, and returns.

```
% Step 2b: State estimate measurement update
r = [xk;zk] - dhat; % innovation: could use to check for sensor errors...
qhat = qhat + L*r;

% Step 2c: Error covariance measurement update
SigmaQ = SigmaQ - L*SigmaD*L';
[~,S,V] = svd(SigmaQ); % Implement Higham method to help robustness
HH = V*S*V';
SigmaQ = (SigmaQ + SigmaQ' + HH + HH')/4;

% Save data in ekfData structure for next time...
spkfData.priorU = uk;
spkfData.priorX = xk;
spkfData.SigmaQ = SigmaQ;
spkfData.qhat = qhat;
bounds = 3*sqrt(diag(SigmaQ));
```


Octave code for the output equation

- The final code segment implements the output equation:

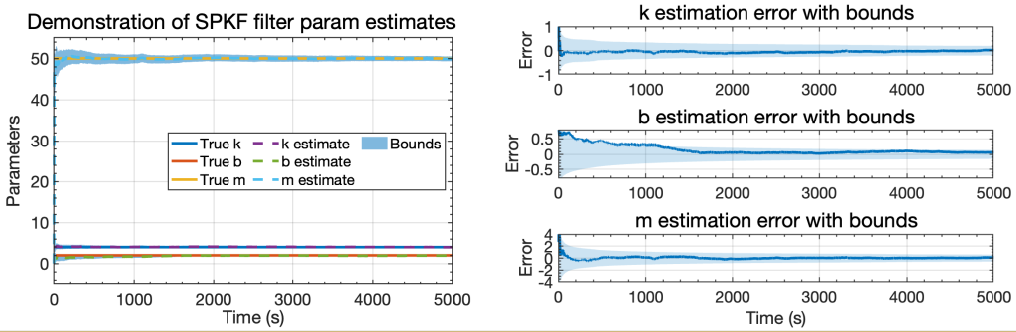
```
% Calculate [xk;zk] for all of parameter vectors in qhat
function dhat = outputEqn(qhat,priorU,xk,priorX,dT,dnoise)
  % Parameters (in order) are k, b, m

  dhat = zeros(length(xk)+1,size(qhat,2));
  for kk = 1:size(qhat,2)
    k = qhat(1,kk); b = qhat(2,kk); m = qhat(3,kk);
    A = [0 1; -k/m -b/m]; % Specify continuous-time model
    B = [0; 1/m];
    C = [1 0];
    Ad = expm(A*dT); Bd = (A\(Ad-eye(2)))*B;

    dhat(:,kk) = [Ad*priorX + Bd*priorU; C*xk]; % [xk;zk] prediction
  end
  dhat = dhat + dnoise;
end
end
```

Sample results

- Some sample results are shown below: the filter is able to converge to the neighborhood of the k estimate very quickly; m takes a little longer; and b takes the longest.
- Convergence speed vs. estimate uncertainty can be tuned by modifying `SigmaW`.



Summary

- You have learned how to implement SPKF code for parameter estimation, assuming that the state vector is known exactly at all points in time.
- The structure is similar to the code used for state estimation: there is a wrapper function, an initialization function, and an iteration function.
- The details of the iteration function change the most, to conform with the steps derived in the previous lesson.
- You have seen that results of parameter estimation, when applied to the linear spring-mass-damper model look very good.